LAPORAN PRAKTIKUM

STRUKTUR DATA

IMPLEMENTASI *ABSTRACT DATA TYPE (ADT)*

MENGGUNAKAN *ARRAYLIST* DALAM PEMROGRAMAN *JAVA*

Disusun Oleh:

Haliya Isma Husna Putri Ahmadi
2511532002

Dosen Pengampu:
Wahyudi. Dr. S.T.M.T

Asisten Praktikum:
Muhammad Galid

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji dan Syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas Rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum Struktur Data pekan 2 dengan judul "Implementasi *Abstract Data Type (ADT)* Menggunakan *ArrayList* dalam Pemrograman Java" tepat pada waktunya.

Penyusunan laporan ini bertujuan untuk memenuhi salah satu tugas dalam mata kuliah Praktikum Struktur Data, sekaligus sebagai sarana pembelajaran bagi penulis dalam memahami konsep *Abstract Data Type (ADT)* serta penerapannya dalam pemrograman Java. Melalui praktikum ini, penulis mempelajari bagaimana mengelola data mahasiswa menggunakan struktur data *ArrayList* dengan menerapkan konsep *ADT*, serta mengimplementasikan operasi seperti penambahan data, penyisipan data, pengubahan data, penghapusan data, dan pencarian data.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat terbuka terhadap kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Ucapan terima kasih yang sebesar-besarnya penulis sampaikan kepada dosen pengampu, asisten praktikum, serta semua pihak yang terlibat dalam pelaksanaan praktikum dan penyusunan laporan ini. Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi penulis maupun pembaca.

Padang, 13 April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR ..... **ii**
DAFTAR ISI ..... **iii**
BAB I PENDAHULUAN **4**
1.1 Latar Belakang 4
1.2 Tujuan 4
1.3 Manfaat 5
BAB II PEMBAHASAN **6**
2.1 Dasar Teori 6
2.1.1 Abstract Data Type (ADT). **6**
2.1.2 Konsep Kerja Abstract Data Type (ADT) **6**
2.1.3* Komponen dalam *Abstract Data Type (ADT) **7**
2.1.4 Primitif Type* dalam *Abstract Data Type (ADT) **8**
2.1.5 ArrayList **9**
2.2 Langkah Kerja 10
2.2.1 Pembuatan *Package* dan *Class* **10**
2.2.2 Membuat Program Mahasiswa_2511532002 ..... **13**
2.2.3 Membuat Program DaftarKata_2511532002 ..... **14**
2.2.4 Membuat Program *ArrayList1_2511532002* **16**
2.2.5 Membuat Program MahasiswaDriver_2511532002 ..... **18**
2.2.6 Membuat Program DaftarKataDriver_2511532002 ..... **22**
2.2.7 *Upload* ke GitHub **24**
2.3 Hasil 26
2.3.1 Hasil Arraylist1_2511532002 ..... **26**
2.3.2 Hasil DaftarKataDriver_2511532002 ..... **26**
2.3.3 Hasil MahasiswaDriver_2511532002 ..... **27**
BAB III KESIMPULAN **29**
3.1 Kesimpulan 29
3.2 Saran 29
DAFTAR PUSTAKA **30**

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan pemrograman, pengolahan data yang terstruktur sangat penting untuk menghasilkan program yang efisien, terorganisir, dan mudah dikembangkan. Salah satu konsep yang digunakan dalam struktur data adalah *Abstract Data Type (ADT*), yaitu suatu tipe data abstrak yang mendefinisikan data beserta operasi-operasi yang dapat dilakukan terhadap data tersebut tanpa memperhatikan implementasi internalnya.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami cara merancang dan mengimplementasikan *Abstract Data Type (ADT)* menggunakan bahasa pemrograman Java, mulai dari pembuatan tipe data objek (*class*), penentuan atribut seperti jam, menit, dan detik, hingga implementasi berbagai operasi seperti validasi, konversi ke detik, operasi aritmatika, serta operasi relasional untuk mengatur interaksi dan pengolahan data waktu dalam program.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

1.2.1 Memahami konsep dasar *Abstract Data Type (ADT)* dalam struktur data.

1.2.2 Mampu mengimplementasikan *Abstract Data Type (ADT)* menggunakan *ArrayList* dalam bahasa pemrograman *Java.*

1.2.3 Mampu membuat *ADT* Jam beserta operasi-operasinya seperti validasi, konversi, aritmatika, dan relasional

1.3 Manfaat

Manfaat yang diharapkan dari pelaksanaan praktikum ini antara lain:

1.3.1 Menambah pemahaman mengenai konsep *ADT* dalam pemrograman.

1.3.2 Meningkatkan kemampuan dalam mengimplementasikan struktur data secara terstruktur.

1.3.3 Membantu dalam memahami pengolahan data waktu menggunakan pendekatan pemrograman.

1.3.4 Menjadi dasar dalam pengembangan program yang lebih kompleks di masa yang akan datang.

BAB II
PEMBAHASAN

2.1 Dasar Teori

Struktur data merupakan salah satu konsep penting dalam ilmu komputer yang digunakan untuk mengorganisasi dan menyimpan data agar dapat diakses dan diolah secara efisien. Dalam pengembangan program, diperlukan suatu cara untuk merepresentasikan data beserta operasi-operasi yang dapat dilakukan terhadapnya secara terstruktur. Salah satu pendekatan yang digunakan adalah *Abstract Data Type (ADT).*

ADT memungkinkan programmer untuk fokus pada fungsi dan perilaku suatu data tanpa harus langsung memikirkan bagaimana data tersebut diimplementasikan di dalam memori. Dengan demikian, penggunaan *ADT* dapat meningkatkan modularitas, keterbacaan, serta kemudahan dalam pengembangan dan pemeliharaan program nantinya.

2.1.1 Abstract Data Type (ADT).

Abstract Data Type (ADT) merupakan suatu konsep yang mendefinisikan sebuah tipe data beserta sekumpulan operasi (primitif) yang dapat dilakukan terhadap tipe data tersebut tanpa memperhatikan bagaimana implementasi internalnya . Dengan kata lain, *ADT* lebih menekankan pada apa yang dapat dilakukan oleh suatu data, bukan bagaimana data tersebut direpresentasikan secara fisik di dalam memori.

2.1.2 Konsep Kerja Abstract Data Type (ADT)

Konsep kerja ADT bersifat modular, di mana ADT berperan sebagai *supplier* (penyedia) yang menyediakan tipe data dan operasi-operasinya, sedangkan program lain bertindak sebagai *client* (pengguna) yang memanfaatkan ADT tersebut.

a) *Supplier* (penyedia)
   Menyediakan tipe data dan operasi-operasinya
b) *Client* (pengguna)
   Menggunakan *ADT* tanpa perlu mengetahui detail implementasinya

Hal ini memungkinkan pemisahan yang jelas antara spesifikasi dan implementasi, sehingga program menjadi lebih mudah dikembangkan, diuji, dan dipelihara.

Dalam implementasinya, ADT biasanya dibagi menjadi:

1) Spesifikasi
   Berisi definisi tipe data dan operasi
2) *Body*/Implementasi
   Berisi kode program dari operasi
3) Driver Program
   Digunakan untuk menguji *ADT*

2.1.3 Komponen dalam *Abstract Data Type (ADT)*

Sebuah *ADT* umumnya terdiri dari beberapa komponen utama, yaitu:

a) Tipe Data (*Type*)
   Merupakan definisi struktur data yang digunakan, misalnya record atau class yang berisi atribut-atribut tertentu.
b) *Primitif* (Operasi Dasar)Primitif adalah operasi-operasi yang dapat dilakukan terhadap ADT, meliputi:
   1) Konstruktor/Kreator
      Untuk membuat objek.
   2) Selektor (*Getter*)
      Untuk mengambil nilai atribut
   3) Mutator (*Setter*)
      Untuk mengubah nilai atribut

4) Validator

Untuk memastikan data valid

5) Operator Relasional

Untuk membandingkan dua objek

6) Operator Aritmatika

Untuk melakukan perhitungan

7) Konversi

Untuk mengubah tipe data ke bentuk lain

c) *Invariant*

Aturan atau batasan yang harus selalu dipenuhi oleh suatu objek agar tetap valid.

d) Aksioma

Aksioma merupakan aturan atau definisi formal yang menjelaskan hubungan logis antar operasi dalam *ADT*. Aksioma digunakan untuk memastikan bahwa setiap operasi yang dilakukan memberikan hasil yang konsisten dan sesuai dengan spesifikasi.

2.1.4 Primitif Type dalam *Abstract Data Type (ADT)*

Primitif type dalam *ADT* adalah sekumpulan operasi dasar yang didefinisikan untuk memanipulasi tipe data tersebut. Primitif ini biasanya diimplementasikan dalam bentuk fungsi atau prosedur.

Beberapa jenis primitif yang umum digunakan antara lain:

1) *IsValid*

Untuk mengecek validitas data.

2) *Make/Create*

Untuk membentuk objek baru.

3) *Get*

Untuk mengambil nilai komponen.

4) *Set*

Untuk mengubah nilai komponen.

5) *Compare*

   Untuk membandingkan dua objek.

6) Operasi tambahan

   Seperti penjumlahan, pengurangan, atau manipulasi lainnya sesuai kebutuhan.

Primitif-primitif ini menjadi inti dari sebuah *ADT* karena menentukan bagaimana data dapat digunakan dan dimanipulasi dalam program.

2.1.5 ArrayList

ArrayList adalah kelas dalam *Java* yang digunakan untuk menyimpan kumpulan data secara dinamis. *ArrayList* adalah bentuk implementasi dari *List.*

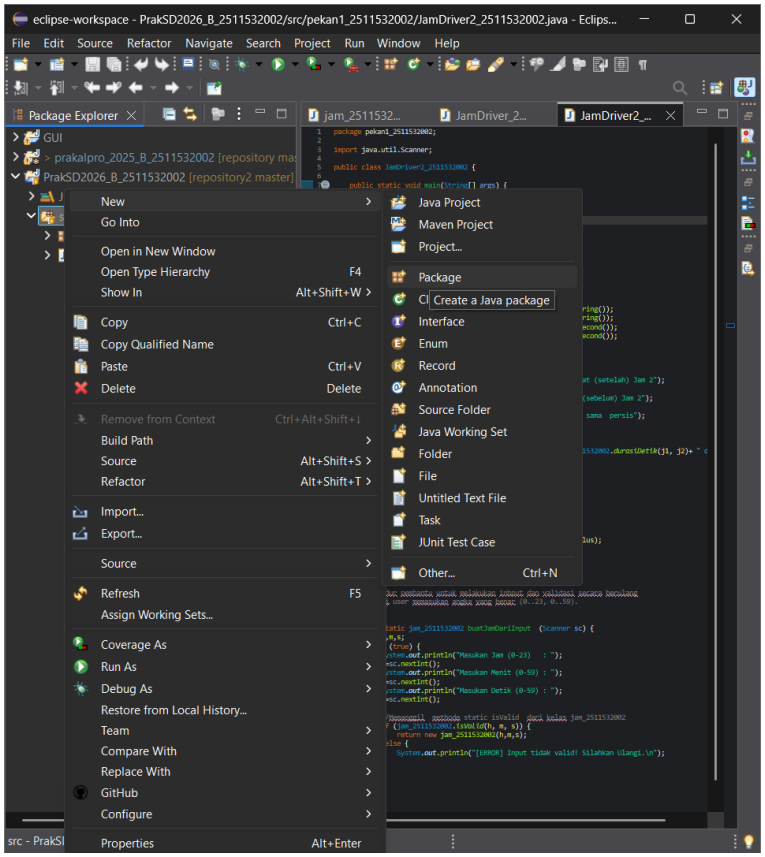
Beberapa sifat penting ArrayList:

1. Ukuran dinamis (bisa bertambah/berkurang otomatis),
2. Menyimpan data secara berurutan (*ordered*),
3. Bisa mengakses data dengan *index,*
4. Bisa menyimpan duplikasi data,
5. Bisa menyimpan nilai *nul*.

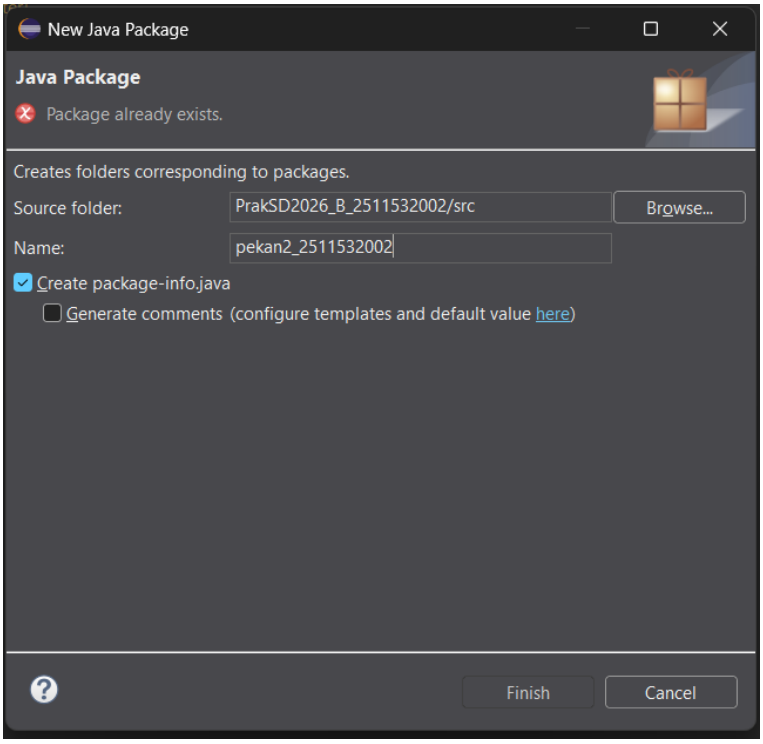
2.2 Langkah Kerja

2.2.1 Pembuatan *Package* dan *Class*

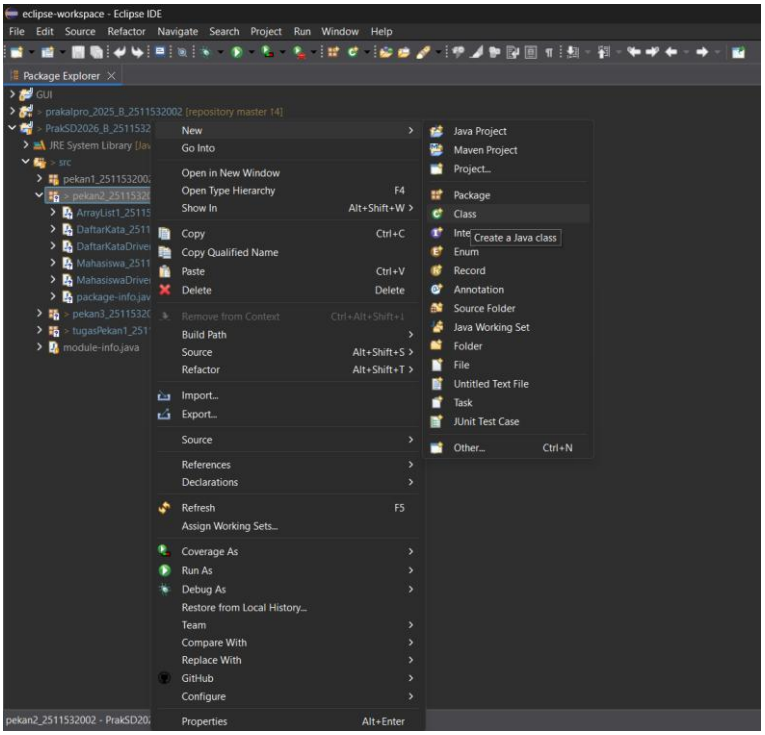
1) Sebelum membuat suatu program *user* perlu untuk membuat *package* terlebih dahulu. Klik kanan pada *src* pada *repository* *"PrakSD2026_B_2511532002"* lalu pilih "*New*" dan klik "*Package*".



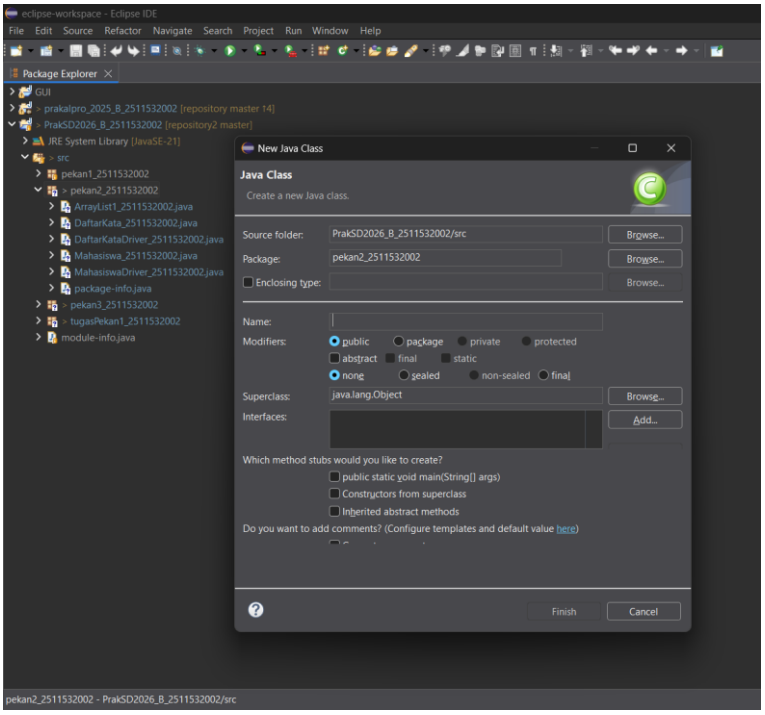
2) Setelah itu akan muncul *pop up* "*Java Package*", buat nama *package* dengan ketentuan tanpa *capslock, space*, dan karakter khusus lainnya. Lalu klik "*finnish*".



3) Selajutnya adalah membuat membuat *class*. pada package yang telah dibuat tapi klik kanan lalu pilih "*New*" dan klik "*Class*".



4) Setelah itu pada menu "*Java Class*" buat nama *class*, dengan ketentuan nama harus capslock diawal kalimat dan tanpa spasi.



2.2.2 Membuat Program Mahasiswa_2511532002

1. Setelah membuat *class Mahasiswa_2511532002,* maka mahasiswa dapat menulis *syntax* nya.

```
Mahasiswa_2511532002.java
1  package pekan2_2511532002;
2
3  public class Mahasiswa_2511532002 {
4      String nim;
5      String nama;
6      String prodi;
7
8      Mahasiswa_2511532002(String nim, String nama, String prodi){
9          this.nim=nim;
10         this.nama=nama;
11         this.prodi=prodi;
12     }
13     @Override
14     public String toString() {
15         return "NIM:" +nim+ ", Nama: "+nama+ ", Prodi: "+prodi;
16     }
17 }
```

2. Mendefinisikan atribut (*Type)*/ variabel data.
   String nim;
   String nama;
   String prodi;

3. Mengisi nilai awal objek saat dibuat dengan konstruktor.
   Mahasiswa_2511532002(String nim, String nama, String prodi){
   this.nim=nim;
   this.nama=nama;
   this.prodi=prodi;
   }

4. Methode *toString()* untuk mebubah objek jadi *String* saat dikeluarkan.
   @Override
   public String toString() {
   return "NIM:" +nim+ ", Nama: "+nama+ ", Prodi: "+prodi;
   }

2.2.3 Membuat Program DaftarKata_2511532002

Mahasiswa_2511532002.java | DaftarKata_2511532002.java

```java
package pekan2_2511532002;

import java.util.ArrayList;

public class DaftarKata_2511532002 {
    private final ArrayList<String>data;

    //Konstruktor: inisiasi list kosong
    public DaftarKata_2511532002() {
        this.data=new ArrayList<>();
    }
    /** Menambahkan elemen di akhir list.*/
    public void tambah (String elemen) {
        data.add(elemen);
    }
    /** Menambahkan elemen pada indeks tertentu (menyisipkan)*/
    public void tambahPada (int index, String elemen) {
        data.add(index, elemen);
    }
    /**
     * Mengubah elemen pada posisi 'index' menjadi 'nilaiBaru'.
     * Bertindak sebagai 'setter' untuk elemen tertentu.
     */
    public void ubahElemen_2511532002(int index, String nilaiBaru) {
        data.set(index, nilaiBaru);
    }
    /**
     * Menghapus elemen pada posisi 'index' dan mengembalikan nilai yang dihapus.
     */
    public String hapusElemen_2511532002(int index) {
        return data.remove(index);
    }
    /**
     * Melakukan iterasi daan mencetak setiap elemen dalam format: [index] nilai
     * (Metode ini tidak mengembalikan nilai; hanya demonstrasi iterasi).
     */
    public void iterasiCetak_2511532002() {
        for (int i=0; i<data.size();i++) {
            System.out.println(data.get(i)+" ");
        }
    }
    /** Mengambil elemen berdasarkan index.*/
    public String get (int index) {
        return data.get(index);
    }

    //Representasi string agar mudah di cetak
    @Override
    public String toString() {
        return data.toString();
    }
    }
```

1. Menentukan atribut *(Type)* yang digunakan untuk menyimpan kumpulan data kata. Dimana atribut "*data*" bertipe *Arraylist* digunakan untuk menyimpan elemen berupa kata.

   public class DaftarKata_2511532002 {
   private final ArrayList<String>data;

2. Membuat konstruktor untuk menginisialisasi *ArrayList*

   public DaftarKata_2511532002() {
   this.data=new ArrayList<>();

3. Membuat *methode* untuk menambahkan data ke dalam *list.*

   Untuk menambahkan data pada akhir list.

   public void tambah (String elemen) {
   data.add(elemen);

Untuk menambahkan pada indeks tertentu dalam list.

```
public void tambahPada (int index, String elemen) {
        data.add(index, elemen);
```

4. *Methode* untuk mengubah dalam *list.* mengganti elemen pada indeks tertentu dengan "*nilaiBaru*", sehingga bertindak sebagai *setter* untuk elemen dalam *list.*

```
public void ubahElemen_2511532002(int index, String nilaiBaru) {
        data.set(index, nilaiBaru);
```

5. *Methode* untuk menghapus elemen dari list.

```
public String hapusElemen_2511532002(int index) {
        return data.remove(index);
```

6. *Methode* untuk mengambil elemen dari list.

```
public String get (int index) {
        return data.get(index);
    }
```

7. *Methode* untuk operasi iterasi dimana menampilkan semua elemen dalam list dengan perulangan.

```
public void iterasiCetak_2511532002() {
        for (int i=0; i<data.size();i++) {
            System.out.println(data.get(i)+" ");
        }
```

8. Methode *toString()* untuk mebubah objek jadi *String* saat dikeluarkan.

```
@Override
    public String toString() {
        return data.toString();
        }
    }
```

2.2.4 Membuat Program *ArrayList1_2511532002*

```java
package pekan2_2511532002;

import java.util.ArrayList;

public class ArrayList1_2511532002 {

    public static void main(String[] args) {

        //size of time ArrayList
        int n=5;

        //Declaring the ArrayList with intitial size n
        ArrayList<Integer> arrli=new ArrayList<Integer>(n);

        //Appending new elements at the end of the list
        for (int i =1; i<= n; i++)
            arrli.add(i);

        System.out.println(arrli);

        //remove element at index (3)
        arrli.remove (3);

        //displaying the ArrayList after deletion
        System.out.println(arrli);

        //printing elements one by one
        for (int i =0; i<arrli.size (); i++)
            System.out.print(arrli.get(i)+" ");
    }
}
```

1. Mendefinisikan variabel *n* dan struktur data yang digunakan yaitu *ArrayList* dengan tipe penyimpanan data bertipe *integrer.*
2. Membuat *ArrayList* dengan kapasitas awal *sebesar n.*

```java
package pekan2_2511532002;

import java.util.ArrayList;

public class ArrayList1_2511532002 {

    public static void main(String[] args) {

        //size of time ArrayList
        int n=5;

        //Declaring the ArrayList with intitial size n
        ArrayList<Integer> arrli=new ArrayList<Integer>(n);
```

3. Menggunakan perulangan *for* untuk menambahkan elemen dari 1 sampai *n* ke dalam *ArrayList* dengan *method add().* Serta menampilkan seluruh isi *ArrayList* secara langsung.

```
for (int i =1; i<= n; i++)
                    arrli.add(i);
            System.out.println(arrli);
```

4. *Methode remove* untuk menghapus elemen dalam *ArrayList* menggunakan *indeks.*

```
arrli.remove (3);
```

5. Menampilkan kembali isi *ArrayList* setelah dilakukan operasi penghapusan.

```
System.out.println(arrli);
```

6. Mengaksees dan menampilkan kembali elemen dengan *methode get* dan perulangan.

```
for (int i =0; i<arrli.size (); i++)
                    System.out.print(arrli.get(i)+" ");
        }
}
```

2.2.5 Membuat Program MahasiswaDriver_2511532002

```java
import java.util.Scanner;

public class MahasiswaDriver_2511532002 {

    //1.Methode untuk menampilkan menu
    public static void tampilkanMenu_2511532002() {
        System.out.println("\nMenu_2511532002");
        System.out.println("1. Tambah Mahasiswa");
        System.out.println("2. Tampilkan Semua Mahasiswa");
        System.out.println("3. Hapus Mahasiswa berdasarkan NIM");
        System.out.println("4. Cari Mahasiswa Berdasarkan NIM");
        System.out.println("5. Keluar");
    }
    // 2. Menambah data baru
    public static void tambahMahasiswa_2511532002(ArrayList <Mahasiswa_2511532002> list, Scanner sc) {
        System.out.println("Masukan NIM: ");
        String nim=sc.nextLine();
        System.out.println("Masukan Nama: ");
        String nama=sc.nextLine();
        System.out.println("Masukan Prodi: ");
        String prodi=sc.nextLine();
        list.add(new Mahasiswa_2511532002 (nim, nama, prodi));
        System.out.println("Mahasiswa berhasil ditambahkan!");
    }
    // 3. Menampilkan semua mahasiswa data
    public static void tampilkanSemuaMahasiswa_2511532002(ArrayList<Mahasiswa_2511532002> list) {
        if (list.isEmpty()) {
            System.out.println("Daftar mahasiswa kosong.");
        }else {
            System.out.println("Data Mahasiswa: ");
            for (Mahasiswa_2511532002 mhs:list) {
                System.out.println(mhs);
            }
        }
    }

    // 4. Menghapus mahasiswa berdasarkan NIM
    public static void hapusMahasiswa_2511532002 (ArrayList<Mahasiswa_2511532002>list, Scanner sc) {
        System.out.println("Masukan NIM yang akan dihapus: ");
        String nimHapus=sc.nextLine();
        boolean removed=list.removeIf(mhs-> mhs.nim.equals(nimHapus));

        if (removed) {
            System.out.println("Data dengan NIM "+nimHapus+" berhasil dihapus.");
        }else {
            System.out.println("NIM tidak ditemukan.");
        }
    }


    // 5. Methode untuk cari Mahasiswa berdasarkan NIM
    public static void cariMahasiswa_2511532002 (ArrayList<Mahasiswa_2511532002>list, Scanner sc) {
        System.out.println("Masukan NIM yang dicari: ");
        String nimCari = sc.nextLine();
        boolean ditemukan =false;

        for (Mahasiswa_2511532002 mhs:list) {
            if (mhs.nim.equals(nimCari)) {
                System.out.println("Hasil Pencarian: ");
                ditemukan=true;
                break;
            }
        }
        if (!ditemukan) {
            System.out.println("NIM tidak ada!");
        }
    }
    public static void main(String[] args) {
        ArrayList<Mahasiswa_2511532002>mahasiswaList_2511532002=new ArrayList<>();
        Scanner scanner=new Scanner(System.in);
        int choice;
    do {
        tampilkanMenu_2511532002();
        System.out.print("Pilih menu: " );
        choice= scanner.nextInt();
        scanner.nextLine();
        switch (choice) {
        case 1:
            tambahMahasiswa_2511532002(mahasiswaList_2511532002, scanner);
            break;
        case 2:
            tampilkanSemuaMahasiswa_2511532002(mahasiswaList_2511532002);
            break;
        case 3:
            hapusMahasiswa_2511532002(mahasiswaList_2511532002, scanner);
            break;
        case 4:
            cariMahasiswa_2511532002(mahasiswaList_2511532002, scanner);
            break;
        case 5:
            System.out.println("Keluar dari program.");
            break;
        default:
            System.out.println("Pilihan tidak valid.");
        }
    }while (choice != 5);
    scanner.close();
    }
}
```

Setelah membuat program *ADT* nya sebagai *supplier* nya, sekarang membuat program *client* untuk menguji atau menggunakan *ADT* yang dibuat.

1. *Method* untuk menampilkan pilihan menu kepada pengguna.

```
public static void tampilkanMenu_2511532002() {
        System.out.println("\nMenu_2511532002");
        System.out.println("1. Tambah Mahasiswa");
        System.out.println("2. Tampilkan Semua Mahasiswa");
        System.out.println("3. Hapus Mahasiswa berdasarkan NIM");
        System.out.println("4. Cari Mahasiswa Berdasarkan NIM");
        System.out.println("5. Keluar");
  }
```

2. Membuat *methode* untuk menerima *input* dan menambahkan data ke dalam *ArrayList*.

```
public static void tambahMahasiswa_2511532002(ArrayList
<Mahasiswa_2511532002> list, Scanner sc) {
        System.out.println("Masukan NIM: ");
        String nim=sc.nextLine();
        System.out.println("Masukan Nama: ");
        String nama=sc.nextLine();
        System.out.println("Masukan Prodi: ");
        String prodi=sc.nextLine();
        list.add(new Mahasiswa_2511532002 (nim, nama, prodi));
        System.out.println("Mahasiswa berhasil ditambahkan!");
  }
```

3. *Method* untuk menampilkan seluruh data mahasiswa. Dan jika kosong, akan ditampilkan pesan.

```
public static void
tampilkanSemuaMahasiswa_2511532002(ArrayList<Mahasiswa_251153
2002> list) {
        if (list.isEmpty()) {
                System.out.println("Daftar mahasiswa kosong.");
        }else {
                System.out.println("Data Mahasiswa: ");
                for (Mahasiswa_2511532002 mhs:list) {
                        System.out.println(mhs);
                }
        }
  }
```

4. Membuat *methode* untuk menghapus data "*Mahasiswa"* berdasarkan *"NIM"*.

```
public static void hapusMahasiswa_2511532002 (ArrayList<Mahasiswa_2511532002>list, Scanner sc) {
        System.out.println("Masukan NIM yang akan dihapus: ");
        String nimHapus=sc.nextLine();
        boolean removed=list.removeIf(mhs-> mhs.nim.equals(nimHapus));

        if (removed) {
                System.out.println("Data dengan NIM "+nimHapus+" berhasil dihapus.");
        }else {
                System.out.println("NIM tidak ditemukan.");
        }
  }
```

5. *Method* untuk mencari mahasiswa berdasarkan "*NIM"* dengan perulangan.

```
public static void cariMahasiswa_2511532002 (ArrayList<Mahasiswa_2511532002>list, Scanner sc) {
        System.out.println("Masukan NIM yang dicari: ");
        String nimCari = sc.nextLine();
        boolean ditemukan =false;

        for (Mahasiswa_2511532002 mhs:list) {
                if (mhs.nim.equals(nimCari)) {
                        System.out.println("Hasil Pencarian: ");
                        ditemukan=true;
                        break;
                }
        }
        if (!ditemukan) {
                System.out.println("NIM tidak ada!");
        }
  }
```

6. Membuat *methode main* sebagai program utama.

```
public static void main(String[] args) {

  ArrayList<Mahasiswa_2511532002>mahasiswaList_2511532002=new ArrayList<>();
        Scanner scanner=new Scanner(System.in);
        int choice;
  do {
        tampilkanMenu_2511532002();
        System.out.print("Pilih menu: " );
        choice= scanner.nextInt();
        scanner.nextLine();
        switch (choice) {
```

```
            case 1:
            tambahMahasiswa_2511532002(mahasiswaList_2511532002,
scanner);
                        break;
            case 2:
   tampilkanSemuaMahasiswa_2511532002(mahasiswaList_2511532002)
;
                        break;
            case 3:
            hapusMahasiswa_2511532002(mahasiswaList_2511532002,
scanner);
                        break;
            case 4:
            cariMahasiswa_2511532002(mahasiswaList_2511532002,
scanner);
                        break;
            case 5:
                        System.out.println("Keluar dari program.");
                        break;
            default:
                        System.out.println("Pilihan tidak valid.");
            }
   }while (choice != 5);
   scanner.close();
   }
}
```

2.2.6 Membuat Program DaftarKataDriver_2511532002

Setelah membuat program *ADT* nya sebagai *supplier* nya, sekarang membuat program *client* untuk menguji atau menggunakan *ADT* yang dibuat.

```
MahasiswaDri...   DaftarKataD...   ArrayList1_...

1  package pekan2_2511532002;
2
3  public class DaftarKataDriver_2511532002 {
4
5      public static void main(String[] args) {
6          DaftarKata_2511532002 al=new DaftarKata_2511532002();
7
8          //Menambah elemen (akhir)
9          al.tambah("Kami");
10         al.tambah("Informatika");
11
12         //Menyisipkan elemen pada iendex 1
13         al.tambahPada(1,"Mahasiswa");
14
15         //Cetak isi awal
16         System.out.println("Awal     : "+al);
17
18         //Mengubah elemen index 1
19         al.ubahElemen_2511532002(1, "Department");
20         System.out.println("Setelah diubah : "+al);
21
22         //Menghapus elemen (hapus index 0)
23         String terhapus=al.hapusElemen_2511532002(0);
24         System.out.println("Terhapus  : " +terhapus);
25         System.out.println("Setelah dihapus : "+al);
26
27         //Iterasi pada ArrayList (cetak setiap elemen)
28         System.out.println("Iterasi : ");
29         al.iterasiCetak_2511532002();
30         System.out.println();
31     }
32
33 }
```

1. Menginisialisasi objek dari *class "DaftarKata"*.
   DaftarKata_2511532002 al=new DaftarKata_2511532002();

2. *Method tambah()* digunakan untuk menambahkan elemen ke akhir list.
   al.tambah("Kami");
   al.tambah("Informatika");

3. *Method tambahPada*() digunakan untuk menyisipkan elemen pada indeks tertentu, sehingga posisi data dapat diatur.
   al.tambahPada(1,"Mahasiswa");

4. Menampilkan seluruh isi *list* menggunakan *method toString*() yang sudah *dioverride* pada *class ADT*.
   System.out.println("Awal   : "+al);

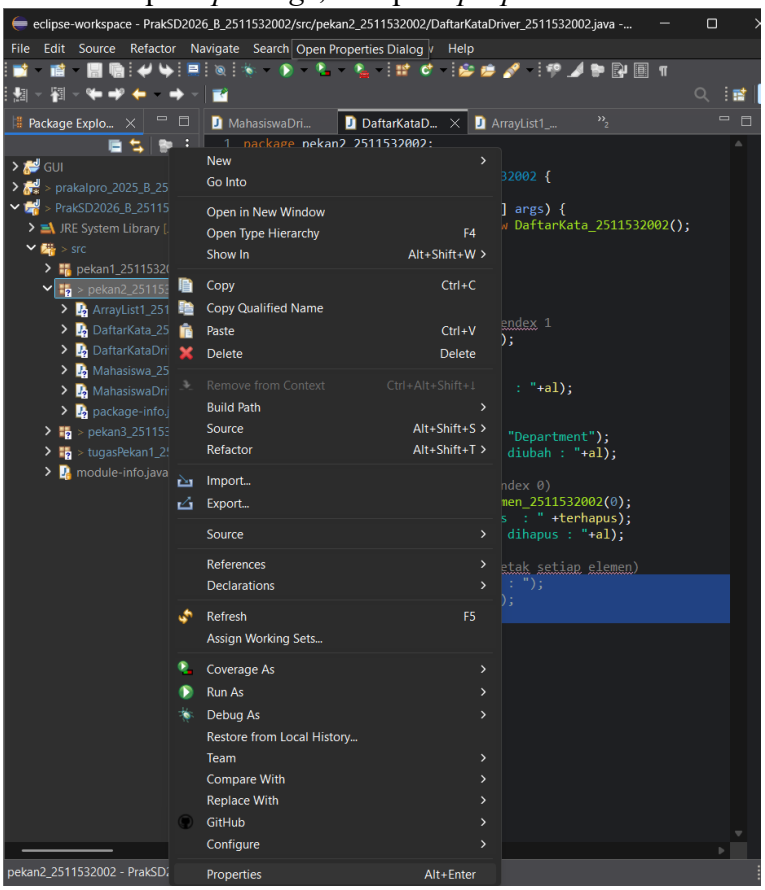
5. *Method ubahElemen*() digunakan untuk mengganti elemen pada indeks tertentu dengan nilai baru.
```
al.ubahElemen_2511532002(1, "Department");
System.out.println("Setelah diubah : "+al);
```

6. *Method hapusElemen*() digunakan untuk menghapus elemen berdasarkan indeks dan mengembalikan nilai yang dihapus.
```
String terhapus=al.hapusElemen_2511532002(0);
System.out.println("Terhapus  : " +terhapus);
System.out.println("Setelah dihapus : "+al);
```

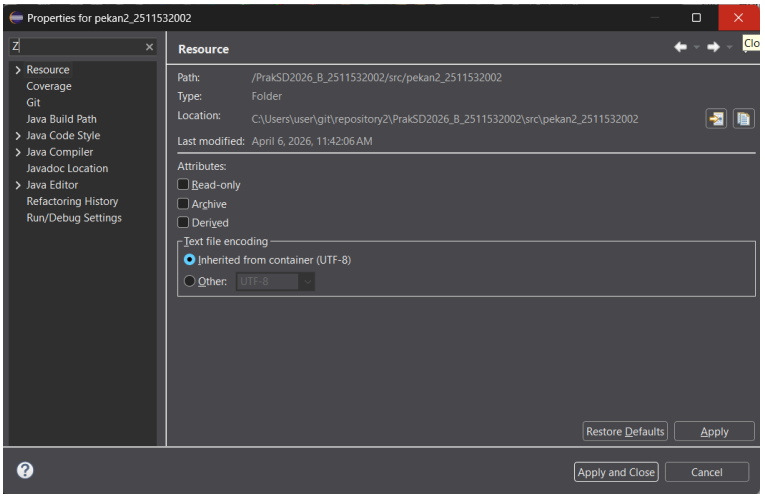
7. *Method iterasiCetak*() digunakan untuk mencetak setiap elemen dalam list satu per satu menggunakan perulangan.
```
System.out.println("Iterasi : ");
al.iterasiCetak_2511532002();
System.out.println();
  }
```

2.2.7 *Upload* ke GitHub

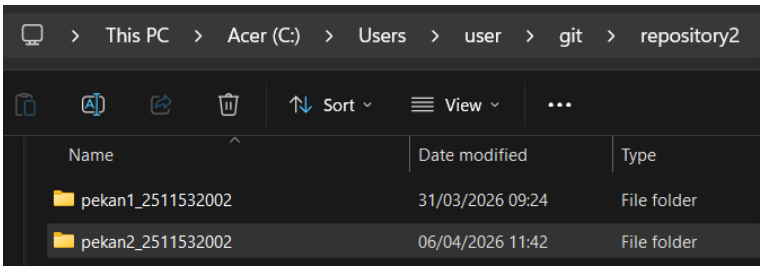
1. Setelah semua program selesai dibuat, selanjutnya adalah *user* perlu untuk memasukan program yang dibuat di Eclips ke *repository* GitHub. Selain dengan cara *"Team"* atau *"Commit and Push"*. Bisa juga dengan cara *upload* manual. Caranya klik kanan pada *package,* lalu pilih *properties.*



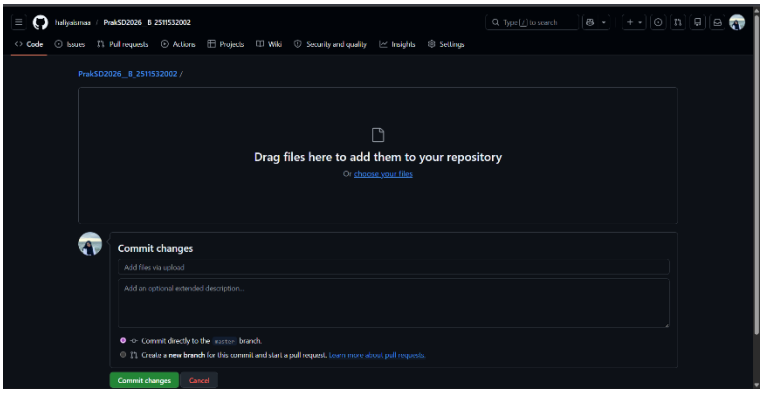
2. Maka akan muncul *pop up menu "Properteis for pekan2_2511532002"*. Lalu klik *icon "Show in System Folder"*.



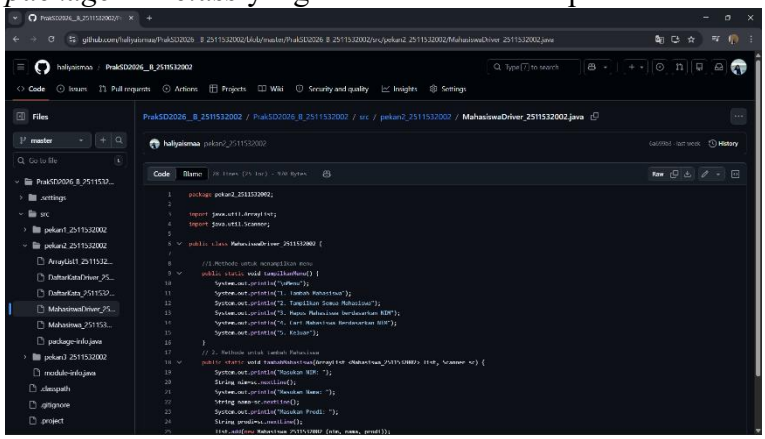
3. Maka akan menunjukan lokasi *file "pekan2_2511532002"* pada *File Explorer computer.*



4. Setelah itu *drag and drop file* "*pekan1_2511532002*" ke GitHub. Lalu tambahkan keterangan jika perlu. Setelah itu klik *upload.*



5. Maka dapat dilihat pada *repository GitHub* bahwa file "*pekan2_2511532002*" telah terunggah via *Upload.*
Dapat kita lihat disini jika kita buka, maka akan muncul *file package* dan *class* yang telah di buat di Eclipse.



2.3 Hasil

2.3.1 Hasil Arraylist1_2511532002

Berikut merupakan hasil akhir dari *ADT jam_2511532002.*

```
Console
<terminated> ArrayList1_2511532002
[1, 2, 3, 4, 5]
[1, 2, 3, 5]
1 2 3 5
```

2.3.2 Hasil DaftarKataDriver_2511532002

Berikut merupakan hasil akhir dari program *DaftarKataDriver_2511532002.*

```
<terminated> DaftarKataDriver_2511532002 [Java Application]
Awal     : [Kami, Mahasiswa, Informatika]
Setelah diubah : [Kami, Department, Informatika]
Terhapus  : Kami
Setelah dihapus : [Department, Informatika]
Iterasi :
Department
Informatika
```

2.3.3 Hasil MahasiswaDriver_2511532002

Berikut merupakan hasil akhir dari program *MahasiswaDriver_2511532002.*

```
Console
<terminated> MahasiswaDriver_2511532002 [Java Application] C:\

Menu_2511532002
1. Tambah Mahasiswa
2. Tampilkan Semua Mahasiswa
3. Hapus Mahasiswa berdasarkan NIM
4. Cari Mahasiswa Berdasarkan NIM
5. Keluar
Pilih menu: 1
Masukan NIM:
2511532002
Masukan Nama:
Haliya Isma
Masukan Prodi:
Informatika
Mahasiswa berhasil ditambahkan!

Menu_2511532002
1. Tambah Mahasiswa
2. Tampilkan Semua Mahasiswa
3. Hapus Mahasiswa berdasarkan NIM
4. Cari Mahasiswa Berdasarkan NIM
5. Keluar
Pilih menu: 2
Data Mahasiswa:
NIM:2511532002, Nama: Haliya Isma, Prodi: Informatika

Menu_2511532002
1. Tambah Mahasiswa
2. Tampilkan Semua Mahasiswa
3. Hapus Mahasiswa berdasarkan NIM
4. Cari Mahasiswa Berdasarkan NIM
5. Keluar
Pilih menu: 1
Masukan NIM:
2511535002
Masukan Nama:
Farah
Masukan Prodi:
DKV
Mahasiswa berhasil ditambahkan!
```

```
Menu_2511532002
1. Tambah Mahasiswa
2. Tampilkan Semua Mahasiswa
3. Hapus Mahasiswa berdasarkan NIM
4. Cari Mahasiswa Berdasarkan NIM
5. Keluar
Pilih menu: 2
Data Mahasiswa:
NIM:2511532002, Nama: Haliya Isma, Prodi: Informatika
NIM:2511535002, Nama: Farah, Prodi: DKV

Menu_2511532002
1. Tambah Mahasiswa
2. Tampilkan Semua Mahasiswa
3. Hapus Mahasiswa berdasarkan NIM
4. Cari Mahasiswa Berdasarkan NIM
5. Keluar
Pilih menu: 3
Masukan NIM yang akan dihapus:
2511535002
Data dengan NIM 2511535002 berhasil dihapus.

Menu_2511532002
1. Tambah Mahasiswa
2. Tampilkan Semua Mahasiswa
3. Hapus Mahasiswa berdasarkan NIM
4. Cari Mahasiswa Berdasarkan NIM
5. Keluar
Pilih menu: 4
Masukan NIM yang dicari:
2511532002
Hasil Pencarian:

Menu_2511532002
1. Tambah Mahasiswa
2. Tampilkan Semua Mahasiswa
3. Hapus Mahasiswa berdasarkan NIM
4. Cari Mahasiswa Berdasarkan NIM
5. Keluar
Pilih menu: 5
Keluar dari program.
```

BAB III
KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa konsep *Abstract Data Type (ADT)* dapat diterapkan dengan baik dalam pemrograman *Java* melalui pembuatan *ADT* berbasis *ArrayList* untuk pengelolaan data dan penerapan berbagai *methode* lainnya. *ADT* ini mampu merepresentasikan data secara terstruktur serta memudahkan dalam proses manipulasi data seperti penambahan, pengubahan, penghapusan, dan pencarian data.

Selain itu, berbagai operasi seperti operasi *CRUD (Create, Read, Update, Delete)* pada *ArrayList* serta penggunaan method seperti *add(), remove(), set(),* dan *get()* dapat diimplementasikan dengan baik dan berjalan sesuai dengan spesifikasi. Penggunaan driver program juga membantu dalam menguji seluruh fungsi yang telah dibuat, sehingga dapat dipastikan bahwa program bekerja dengan benar.

3.2 Saran

Saran untuk praktikum selanjutnya adalah agar mahasiswa lebih teliti dalam memahami penulisan *syntax* pemrograman Java dan penggunaan struktur data *ArrayList* untuk menghindari kesalahan atau error, terutama dalam penggunaan indeks, pemanggilan *metShod*, serta pengelolaan data dalam program.

DAFTAR PUSTAKA

[1] Inggriani Liem, “Diktat Struktur Data,” Program Studi Teknik Informatika, Institut Teknologi Bandung, 2008.